OVER / PARTITION BY

Sliding Frames & Analytical Vector Partitions

SQL FOUNDRY // ARCHITECTURE MANUAL

Phase 5.5 / 6 (Evaluated AFTER HAVING, right before SELECT)

Complexity: O(N log P) where P is partition size.

1. ANSI Syntax & Grammar Specification

```
function_name([expression [, ...]]) OVER (
    [ PARTITION BY partition_expression [, ...] ]
    [ ORDER BY sort_expression [ ASC | DESC ] [, ...] ]
    [ frame_clause ]
)
```

Unlike GROUP BY which collapses multiple rows into a single summary row, Window Functions compute running totals, rankings, moving averages, and lead/lag offsets while preserving every individual input row. The OVER clause defines the partitioning fences and sliding frame boundaries.

2. Database Engine Internals & Storage Footprint

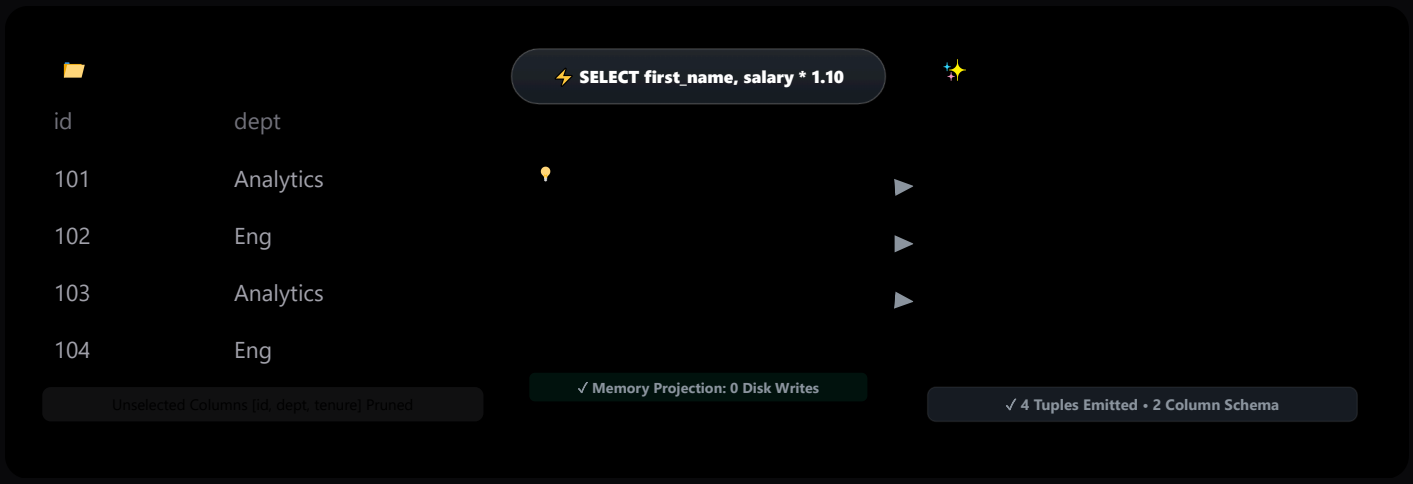
Physical Operator: WindowAgg / Stream Partition Buffer

Memory Footprint: O(P) where P is the size of the current partition. Uses a sliding window ring buffer.

Disk Spill Risk: Moderate. If partition ordering requires sorting and the partition size exceeds work_mem, it spills to temporary disk files.

Index SARGability: An index matching (partition_col, order_col) allows WindowAgg to stream tuples directly with zero sorting or memory buffering.

3. Relational Architecture Diagram (Tuple Lifecycle)



Overview | 1. Scan Table | 2. Relational Engine | 3. Emit Result

OVER / PARTITION BY // IMPLEMENTATION PATTERNS

MODULE: ORDERING & WINDOW FRAMES

PAGE 2 OF 4

4. Canonical Query Code Progression

Pattern A: Row Numbering & Dense Ranking within Department

```
-- Ranking employees by salary within each department
SELECT
    emp_id,
    first_name,
    department,
```

```
    salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS salary_rank
FROM enterprise_employees;
```

Pattern B: Running Total & 7-Day Moving Average

```
-- Running cumulative revenue and 7-day smoothing window
SELECT
    order_date,
    daily_revenue,
    SUM(daily_revenue) OVER (
        ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumulative_running_revenue,
    ROUND(AVG(daily_revenue) OVER (
        ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ), 2) AS moving_avg_7day
FROM daily_revenue_summary;
```

Pattern C: Lead/Lag Delta & Period-over-Period Growth Rate

```
-- Calculating month-over-month revenue delta and growth percentage
WITH monthly_sales AS (
    SELECT
        DATE_TRUNC('month', order_date) AS sales_month,
        SUM(total_amount) AS monthly_revenue
    FROM customer_orders
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
    sales_month,
    monthly_revenue,
    LAG(monthly_revenue, 1) OVER (ORDER BY sales_month) AS prior_month_rev,
    monthly_revenue - LAG(monthly_revenue, 1) OVER (ORDER BY sales_month) AS mom_dollar_growth,
    ROUND(
        ((monthly_revenue - LAG(monthly_revenue, 1) OVER (ORDER BY sales_month))
        / NULLIF(LAG(monthly_revenue, 1) OVER (ORDER BY sales_month), 0)) * 100,
        2
    ) AS mom_growth_pct
FROM monthly_sales;
```

5. Production Schemas & Input/Output Relational Tables

Input: Engineering Partition Tuples

name	department	salary
David	Engineering	$125,000
Sara	Engineering	$110,000
Carlos	Engineering	$72,000

Emitted Output Table

name	salary	rank	running_total
David	$125,000	1	$125,000
Sara	$110,000	2	$235,000
Carlos	$72,000	3	$307,000

6. Production Enterprise Case Study

Company / Team: JPMorgan Chase Quantitative Risk & Treasury

Operational Scenario: Risk quants model daily trading book losses over 500 trading days, isolating the 1st percentile historical loss threshold to meet Basel III market risk capital requirements.

```
WITH ranked_daily_pnl AS (
    SELECT
        trading_date,
        daily_pnl_usd,
        PERCENT_RANK() OVER (ORDER BY daily_pnl_usd ASC) AS pnl_percentile
    FROM trading_book_daily_pnl
    WHERE trading_date >= CURRENT_DATE - INTERVAL '500' DAY
)
SELECT
    MAX(daily_pnl_usd) AS var_99_loss_threshold
FROM ranked_daily_pnl
WHERE pnl_percentile <= 0.01;
```

Business & Infrastructure Impact: Delivers audited CCAR and Basel III Value-at-Risk calculations to senior risk committees daily without requiring external Python/R batch jobs.

7. FAANG & Wall Street Technical Interview Defense

[Google / Meta] Question 1:

What is the critical behavioral difference between RANK(), DENSE_RANK(), and ROW_NUMBER() when ties occur?

Model Answer: ROW_NUMBER() guarantees unique incremental integers (1, 2, 3, 4) regardless of ties. RANK() assigns the same rank to identical values but leaves gaps for subsequent ranks (1, 2, 2, 4). DENSE_RANK() assigns the same rank to ties but does NOT leave gaps (1, 2, 2, 3). In LeetCode #185 (Top 3 Department Earners), DENSE_RANK() is mandatory because the 3rd highest unique salary can be shared by multiple employees.

[Amazon / Apple] Question 2:

What is the default window frame when ORDER BY is specified inside an OVER() clause without an explicit ROWS/RANGE clause?

Model Answer: The ANSI default frame is "RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW". This carries two massive hazards: 1) RANGE groups duplicate values into peers, causing running totals to jump instead of smoothly accumulating row-by-row, and 2) RANGE requires memory spooling and is vastly slower than "ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW".

---

8. High-Yield Pitfalls & Corporate Production Bugs

Critical **The Default RANGE Frame Trap:** Omitting the frame clause when computing a running sum defaults to RANGE, which groups identical timestamps together and sums them in a single lump jump instead of calculating a true cumulative sum.

High **Attempting to filter window functions in WHERE:** Writing "WHERE ROW_NUMBER() OVER (...) <= 3" fails because WHERE executes at Phase 2, long before window functions evaluate at Phase 6. You must wrap the window query in a CTE or subquery (or use QUALIFY in Snowflake/BigQuery).

9. Certification & Conceptual Mastery Examination (5 MCQs)

1. What is the result of DENSE_RANK() for salaries: $100k, $90k, $90k, $80k?

A) 1, 2, 3, 4
B) 1, 2, 2, 4
C) 1, 2, 2, 3
D) 1, 1, 2, 3

Correct Answer: C • *DENSE_RANK() assigns rank 2 to both $90k earners and advances immediately to 3 for $80k without leaving a gap.*

2. Why does "SELECT name, salary FROM emp WHERE RANK() OVER (ORDER BY salary DESC) <= 3" trigger a syntax error?

A) Window functions cannot be used in WHERE clauses because WHERE executes at Phase 2 before window functions evaluate at Phase 6
B) RANK() requires a PARTITION BY clause
C) The syntax requires parentheses around the WHERE clause
D) Salary must be an integer
Correct Answer: A • *Window functions are evaluated at Phase 6, after WHERE has already filtered rows. To filter window ranks, wrap in a CTE or subquery.*
3. What does LAG(salary, 2, 0) OVER (ORDER BY hire_date) return for the second row in the partition?
A) The salary from 2 rows prior, falling back to default value 0 because 2 rows prior does not exist
B) The salary of the current row
C) NULL
D) Syntax error
Correct Answer: A • *LAG(col, offset, default) looks back "offset" rows. For row 2, looking back 2 rows exceeds the partition boundary, so the specified default 0 is returned.*
4. What is the clause available in Snowflake, BigQuery, and DuckDB that filters window function results directly without requiring a CTE wrapper?
A) HAVING
B) QUALIFY
C) FILTER
D) SIFT
Correct Answer: B • *QUALIFY executes at Phase 6.5, filtering window results directly (e.g. QUALIFY ROW_NUMBER() OVER (...) <= 3) without nested CTE boilerplate.*
5. How does PARTITION BY differ from GROUP BY in terms of output row count?
A) PARTITION BY preserves all original input rows; GROUP BY collapses rows into a single summary row per group
B) They are exact synonyms
C) PARTITION BY deletes duplicate rows
D) GROUP BY requires an index
Correct Answer: A • *GROUP BY is a dimensional collapse operator; PARTITION BY computes analytical metrics over subsets while preserving 100% of individual tuples.*